

Uber 设计工程对齐体系 从 Base 到 uSpec 的全链路实践

Uber 用三层系统解决了“数百组件 × 7 个技术栈 × 3 套无障碍 API”规模下的设计-工程同步问题。第一层是 Base 设计系统定义规范，第二层是 uSpec AI Agent 自动将设计翻译为跨平台技术合同，第三层是 Design System Observability 持续验证代码实现。设计师不需要写代码，但必须以结构化、Token 化的方式在 Figma 中工作。

7 实现平台栈 UIKit · SwiftUI · Android XML · Compose · React · Go · SDUI	3x Base 组件开发速度 相比自定义组件	<2 min uSpec 规范生成 此前需要数周	50% 代码量减少 使用 Base 组件 vs 自定义
---	-------------------------------------	--	--

I. 体系全貌：三层架构

1. 为什么需要系统化

Uber Rider app 同时在全球数百个页面上发布功能，使用数千个 feature flag。没有任何设计师、工程师或 QA 能靠人工掌握所有用户流。当设计系统采用率无法量化时，Uber 只能通过用户投诉和公众反馈才知道问题。

2. 三层体系定义

层级	系统	核心功能
第一层	Base 设计系统	定义组件、Token、规范，设计师日常工作的基础
第二层	uSpec AI Agent	自动将 Figma 设计翻译为 7 个技术栈的完整规范
第三层	Design System Observability	自动扫描线上 app，量化设计系统采用率

II. 设计师在 Figma 中的五条规则

1. 只用 Base 组件

核心规则

所有 UI 元素必须使用 Base 设计系统提供的标准组件。Uber 将组件分为三类：Base（合规）、Non-Base（看起来像 Base 但实际是自定义重造的）、Custom（确实没有对应 Base 组件的定制元素）。Non-Base 是最危险的——它具有 Base 的外观但缺少内置的无障碍、主题适配和跨平台一致性。

指标	BASE 组件	自定义组件
开发速度	3x 更快	基线
视觉一致性问题	4x 更少	基线
代码量	减少 50%	基线
主题更新周期	数周	数月

2. 使用 Design Token

核心规则

颜色、间距、字体必须通过 Figma Variables 和 Design Token 指定，不能写死数值。uSpec Agent 直接从 Figma 文件中读取 token 名称来生成技术规范。使用裸 hex 值（如 #000000）而非语义 token（如 contentPrimary）会导致自动化流程断裂。

Base 设计系统的 Foundation 层覆盖：Color System（语义化颜色 token）、Typography（字体尺度 token）、Spacing/Layout（间距栅格 token）、Elevation（阴影层级）、Motion（动画曲线和时长）、Icons/Illustrations（标准化图标库）。

3. 结构化组件与变体命名

Figma 中的组件结构必须清晰规范，以供 uSpec Agent 爬取和理解。以一个 Button 组件为例，以下是 Uber 要求的具体规范：

Variant Axes（变体轴）必须明确定义。每个组件需要在 Figma 中通过 Component Properties 面板创建清晰的变体维度。以 Button 为例，需定义：Size = Small / Medium / Large、State = Default / Hover / Pressed / Disabled / Loading、Style = Primary / Secondary / Tertiary、Width = Hug / Fill。Agent 读取这些变体轴后，自动生成 API 文档中的属性表——每个轴变成一个可配置属性，每个值变成该属性的枚举选项。如果设计师用“复制 + 手动改样式”代替变体轴定义，Agent 将无法识别这些选项之间的关系。

Boolean Toggles 用于开关属性。以 Button 的图标为例，正确做法是在 Component Properties 中添加 hasLeadingIcon = true / false 和 hasTrailingIcon = true / false，而非创建“有图标”和“无图标”两个独立变体。Agent 会将 Boolean Toggle 映射为代码中的可选参数（如 SwiftUI 的 leadingIcon: Image?）。

Slot-based composition（插槽式组合）。以 ListItem 组件为例，左侧的 leading 区域应设为 instance swap 插槽，允许替换为 Avatar、Icon、Checkbox 等不同子组件，而不是在 ListItem 内部硬编码一个固定的 Avatar。这让 Agent 能识别出“这里是一个可替换的插槽”，并在 API 文档中生成 leading: Slot<Component> 类型的参数说明。

图层命名必须遵循语义化规则。Agent 通过 DFS 遍历 Figma 图层树来识别组件的解剖结构（Anatomy）。以 Button 为例：

正确命名	错误命名	AGENT 如何使用
Container	Frame 47	识别为组件的根容器，标注 padding 和 border
Label	Text	标注字体 token、行高、对应的无障碍 label
LeadingIcon	Group 2	识别为可选图标插槽，标注尺寸 token
Spinner	loading copy	识别为 Loading 状态的指示器元素

Figma 默认生成的 `Frame 47`、`Group 2` 等名称对 Agent 来说是噪音。Uber 要求所有图层在设计完成时必须完成语义化重命名。

4. 无障碍信息在设计阶段完成

每个组件需覆盖三套无障碍 API 的语义描述。这是 Uber 设计流程中最耗时的部分。一个属性填错，辅助技术用户在真实设备上就会遇到问题。

平台	框架	设计师需要关注的内容
iOS	VoiceOver	modifiers、traits、label、hint
Android	TalkBack	semantics、contentDescription、role
Web	ARIA	role、aria-label、aria-expanded 等

uSpec 将这一过程从手动交叉引用三套平台文档，简化为：设计师提供上下文意图，Agent 自动从文档化的 API 中选择正确属性值。

5. 密度变体跨平台一致

同一组件在不同屏幕密度下的 padding、字体、高度需通过 Figma Variable Modes 管理。不能手动复制多份设计。Agent 的 Density Logic Skill 会计算 padding 和字体如何在 SwiftUI、React、Compose 等栈中缩放。

III. uSpec：AI Agent 如何工作

1. 架构概览

uSpec 是一个"视觉到技术规范的编译器"。Cursor IDE 中的 AI Agent 通过 WebSocket 连接本地 Figma Desktop，使用开源的 Figma Console MCP 桥接协议读写 Figma 文件。整条流水线完全本地运行，没有云端 API 调用。

- AI Agent（Cursor IDE）→ WebSocket → Figma Console MCP → Figma Desktop（本地）
- 所有 agentic 请求通过 GenAI Gateway（Go 编写），支持 PII 脱敏后才发往 Claude 3.5 / GPT-4o
- 集成于 Uber 的 Michelangelo AI 平台

2. 六大 Agent Skill

每个规范维度由独立的 Agent Skill 负责。Skill 是结构化 Markdown 文件，编码了 Uber 内部的领域专业知识。

SKILL	生成内容
Anatomy	组件实例上的编号标记 + 属性表
API	属性、值、默认值、配置示例
Properties	变体轴和布尔开关的实例预览
Color Annotation	每个元素在每个状态下对应的 token
Structure	尺寸、间距、padding 的跨变体映射
Screen Reader	VoiceOver + TalkBack + ARIA 一次性生成

3. 设计师的实际操作

两步完成：分享 Figma 链接 + 补充上下文。

- 在 Figma 中按规范构建组件（遵循第二节五条规则）
- 将 Figma 组件链接发给 Cursor IDE 中的 Agent
- 补充 Agent 无法推断的上下文——特定状态的行为说明、平台特异性
- 审查 Agent 在 Figma 文件中直接渲染的完整规范页面

Agent 处理的部分：编写 Anatomy 表格、API 文档、颜色标注、Structure 计算、三平台 Screen Reader 规范、格式化排版。

“

"No cloud API, no proprietary design data leaving your network. At Uber, this is what makes AI-assisted documentation possible in the first place: nothing leaves your machine."

Ian Guisard, Uber Design Systems

IV. Design System Observability: 闭环验证

1. Base Counter

DFS 深度优先遍历 view tree，逐个检查 view class 是否属于已知 Base 组件。Debug 模式下，屏幕上每个元素被高亮标记为绿色（Base 组件）或红色（非 Base 组件）。三步流程：

- Trigger：屏幕级导航变化自动触发 Base Counter
- Counting Algorithm：从 root view 开始后序 DFS 遍历，识别 Base 组件类
- Decorator：用着色节点装饰器对 view 节点进行可视化标记

这让数千名不同职能的人都能看到同一个质量指标，而非依赖少数专家。

2. 每日自动化分析

两条互补的自动化管线持续监测数千个页面。

- 广域统计：所有内部测试者自动触发分析事件，聚合众包数据反映真实使用
- 深度截图：利用 E2E 测试套件在每日 CD 构建时截取页面截图，运行详细设计指标分析
- 违规自动创建 Jira ticket，分配给对应页面的设计和工程负责人
- 结合人工 Feature Review Readiness 流程，确保未来开发不降低设计质量

3. 指标升级为业务级

设计系统采用率已与测试覆盖率、延迟、可用性并列为 Uber 的业务影响指标。设计师和工程师看同一套数据，共同目标是 100% Base 合规率。这个过程从 Base Race 激励挑战开始，逐步获得高管支持，最终从每月数小时的人工审计变成了全自动化系统。

V. 设计师的代码理解力要求

1. 不需要写代码

设计师不需要写 Swift、Kotlin 或 React。uSpec 的核心设计让设计师继续在 Figma 中工作，AI 完成所有翻译。设计师的角色从"出图"转向"翻译意图"。

2. 需要理解的概念图谱

以下六个概念是 Uber 设计师的必备知识。每个概念附带具体场景和操作指导。

① Design Token（必须熟练）。Token 是设计属性的语义化别名。Uber 的 Base 系统中，设计师不写 `#276EF1`，而写 `contentAccent`。当品牌升级或暗色模式切换时，token 的值在一处更新，所有引用自动生效。在 Figma 中的操作：选中图层 → 右侧 Fill 面板点击色块 → 在 Libraries 中选择对应的 Color Variable，而非手动输入 hex 值。间距同理：padding 不写 `16px`，而绑定 `spacing-400` 这个 Variable。uSpec Agent 读取的是 token 名称而非数值——裸值会导致 Color Annotation Skill 生成空白。

② Component API (理解概念)。每个 Figma 组件本质上就是一个"API": 它有属性 (Properties)、默认值 (Defaults) 和变体 (Variants)。以 Uber 的 Badge 组件为例, API 可能是 `color = Positive / Negative / Warning / Neutral`, `size = Small / Default`, `hasIcon = true / false`。设计师不需要写代码, 但要理解: 你在 Figma 中定义的 Component Properties 面板就是工程师在代码中看到的 props 接口。如果设计师遗漏了一个变体, 工程师就会缺少对应的实现规范。

③ 无障碍语义 (理解场景意图)。设计师不需要记住 `accessibilityTraits = .button` 这样的代码, 但需要回答: "这个元素是什么? 它当前的状态是什么? 用户操作后会发生什么?" 以 Uber 的"确认上车点"按钮为例, 设计师需要标注: 这是一个按钮 (role), 标签是"确认上车点" (label), 按下后的结果是"确认位置并呼叫司机" (hint)。Agent 会将这些意图自动映射为 VoiceOver 的 `.isButton` trait + `accessibilityLabel`, TalkBack 的 `contentDescription` + `role = Button`, 以及 ARIA 的 `role="button"` + `aria-label`。

④ Figma Variables & Modes (必须熟练)。Uber 用 Variable Modes 管理同一组件在不同密度和主题下的表现。以 Button 为例, 在 Figma 中创建一个 `density` Variable Collection, 定义 `Compact` 和 `Comfortable` 两个 Mode。Compact 模式下 Button 高度绑定 `size-800` (32px), Comfortable 模式下绑定 `size-1200` (48px)。设计师在一个组件上完成设计, 切换 Mode 即可预览不同密度的表现——无需复制两份设计稿。Agent 的 Structure Skill 会读取所有 Mode 的值, 自动生成跨密度的尺寸映射表。

⑤ 组件组合模式 (必须掌握)。Uber 的组件不是扁平的, 而是嵌套组合的。以 Card 组件为例, 它的结构是: Card → Header (内含 Title + Subtitle + TrailingAction slot) → Body (内含 content slot) → Footer (内含 action buttons slot)。`"Slot"`意味着这个位置可以用 Figma 的 instance swap 功能替换为任何兼容组件。设计师必须用 instance swap 暴露这些插槽, 而非把子组件直接画死在 frame 里。这让 Agent 能识别出"这里是一个可替换的插槽", 并在 API 文档中生成类型化的参数说明。

⑥ 平台差异 (了解级别)。同一个 Bottom Sheet 在 iOS 上使用系统级的 `UISheetPresentationController`, 在 Android 上使用 Material 的 `BottomSheetBehavior`, 在 Web 上是自定义的 React modal。设计师不需要了解实现细节, 但需要知道: iOS 的 sheet 有系统级的拖拽手势和 detent (停靠点), Android 的 sheet 默认半展开, Web 的 sheet 没有原生手势。这些差异会影响设计时的交互决策。Agent 的平台映射 Skill 会处理具体的 API 翻译, 但设计师在提供上下文时, 需要说明"这个 sheet 是否需要半展开状态"这类跨平台的交互意图。

3. 设计工程三原则

Uber Design Engineering 负责人 Vietanh Nguyen 总结的三条原则, 每条都有对应的实践案例:

原则一: "Be the Translator between Design and Engineering" (做翻译者, 而非出图者)。在 Uber 的"确认上车点"页面改版中, 设计师发现底部 sheet 和返回按钮的实现与 Figma 交付稿存在细微偏差——肉眼几乎看不出, 但在 Base Counter 扫描中被标记为 Non-Base。设计工程师介入后发现, 工程师使用了自定义 frame 仿造了 Base sheet

的外观，但未调用 Base 组件库的代码。设计师的角色不是"重新出一版设计"，而是准确翻译设计意图：这里应该使用 Base 的 `BottomSheet` 组件，配置 `detent = [.medium, .large]`，而非自定义实现。这种精确的意图翻译避免了工程团队反复猜测。

原则二："Bridge the Gaps via Toolings"（用工具弥合鸿沟，而非用会议）。Uber 早期的做法是每周安排一组设计师坐在会议室里，手动对比 Figma 设计稿和工程实现——逐屏核对颜色、间距、字体是否匹配。这个流程耗时且容易遗漏。Design System Observability 将这个流程自动化：Base Counter 在每日 CD 构建中自动截图并扫描所有页面，发现 Non-Base 元素自动创建 Jira ticket。2024 年初的一次基础设施迁移意外导致部分页面的设计质量退化，正是因为有自动化监测，工程团队得以在用户感知之前快速修复。工具取代了人工会议，且覆盖面从"几个设计师能看到的页面"扩大到"数千个页面"。

原则三："Enable Vision with Leadership and Coding"（用领导力和编码能力实现愿景）。Uber 在推进 Base 设计系统采用时，最大的障碍不是技术，而是跨团队对齐。Vietanh Nguyen 的做法是：先用 Base Race 挑战赛在各团队之间制造竞争和可见性（领导力），同时亲自编写 Base Counter 的核心检测逻辑和 Decorator 可视化工具（编码能力）。他通过数百小时的人工审计建立信任，用数据证明 Base 组件的开发效率提升，最终说服高管将设计系统采用率写入公司级 OKR。这条原则的核心是：设计工程师不能只停留在 Figma 和 Sketch 中，也不能只停留在"倡导"层面——需要同时具备向上说服的领导力和向下落地的代码实现能力。

VI. 对中小团队的启发

即使没有 AI Agent

Uber 体系的核心不是 AI，而是结构化。即使团队没有 uSpec 级别的工具，只要做到以下四点，设计到工程的翻译成本就会大幅下降：

- 用 Design Token 而非裸值——颜色、间距、字体全部 token 化
- 用真正的组件而非视觉仿品——杜绝"看起来像但不是"的 Non-Base 元素
- 将设计系统采用率量化为与工程指标并列的团队目标——让所有人看同一个数字

在 Figma 中建立清晰的图层命名和变体结构，是最值得优先执行的一步。具体操作指南：

图层命名规范。所有图层必须使用语义化名称而非 Figma 默认名称。规则是：名称描述"这个元素是什么"而非"它在哪里"。例如

`Container`、`Label`、`LeadingIcon`、`Divider`，而非 `Frame 47`、`Text`、`Group 2`、`Line 1`。命名使用 PascalCase（首字母大写），与代码中的组件命名保持一致。嵌套层级应反映组件的逻辑结构：`Card → Header → Title`，而非 `Card → Frame 3 → Text`。

变体结构规范。组件的所有可配置维度必须通过 Figma Component Properties 定义，而非通过"复制 + 手动修改"。每个变体轴应使用明确的名称和值：`Size = Small / Medium / Large` 而非 `v1 / v2 / v3`。布尔属性使用 `has` 前缀（如 `hasDivider`、`hasAvatar`）。可替换的子组件使用

instance swap 而非固定嵌套。这些约定即使在没有 AI 工具的团队中，也能让工程师直接从 Figma 的 Properties 面板读取组件的完整 API，减少设计交付时的口头沟通成本。

交付检查清单。在设计交付前，确认以下项目：所有图层已语义化命名（无 `Frame`、`Group`、`Text` 等默认名）；所有颜色、间距、字体已绑定 Variable（Properties 面板中显示变量名而非裸值）；所有变体轴和布尔属性已在 Component Properties 中定义；可替换区域已设置为 instance swap 插槽。这份清单本身就是一个最小可行的"设计质量门禁"。

